AI 실행을 통한 최적의 클러스터 및 GPU 활용률

NetApp Solutions

NetApp
April 06, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/netapp-solutions/ai/osrunai_run_ai_installation.html on April 06, 2022. Always check docs.netapp.com for the latest.

목차

실행 시 클러스터 및 GPU 활용률 최적화: AI . . . 1
AI 설치 를 실행하십시오 . . . 1
실행: AI 대시보드 및 뷰 . . . 1
데이터 과학 팀을 위한 프로젝트 생성 및 GPU 할당 . . . 2
실행 중인 작업 제출: AI CLI . . . 3
높은 클러스터 사용률 달성 . . . 6
덜 까다로운 워크로드 또는 대화형 워크로드에 대한 부분 GPU 할당 . . . 7
할당량 초과 GPU 할당을 통한 높은 클러스터 사용률 달성 . . . 8
기본 자원 할당 공정성 . . . 9
할당량 초과 공정성 . . . 10
Trident에서 프로비저닝한 PersistentVolume에 데이터 저장 . . . 12

실행 시 클러스터 및 GPU 활용률 최적화: AI

다음 섹션에서는 AI 설치, 테스트 시나리오, 그리고 이 검증에서 수행된 결과에 대해 자세히 설명합니다.

TensorFlow 벤치마크를 비롯하여 업계 표준 벤치마크 툴을 사용하여 이 시스템의 운영 및 성능을 검증했습니다. ImageNet 데이터 세트는 이미지 분류를 위해 유명한 CNN(Convolutional Neural Network) DL 모델인 ResNet-50을 교육하는 데 사용되었습니다. ResNet-50은 더욱 빠른 처리 시간으로 정확한 교육 결과를 제공하므로 스토리지에서 충분한 수요를 창출할 수 있습니다.

"다음: AI 설치를 실행합니다".

AI 설치 를 실행하십시오

Run:AI를 설치하려면 다음 단계를 완료하십시오.

1. DeepOps를 사용하여 Kubernetes 클러스터를 설치하고 NetApp 기본 스토리지 클래스를 구성합니다.
2. GPU 노드 준비:
   a. NVIDIA 드라이버가 GPU 노드에 설치되었는지 확인합니다.
   b. nVidia-docker가 기본 Docker 런타임으로 설치 및 구성되어 있는지 확인합니다.
3. 러닝 설치: AI:
   a. 에 로그인합니다 "AI 관리자 UI를 실행합니다" 클러스터를 생성합니다.
   b. 생성된 'runai-operator-<clustername>.yAML' 파일을 다운로드합니다.
   c. Kubernetes 클러스터에 운영자 구성을 적용하십시오.

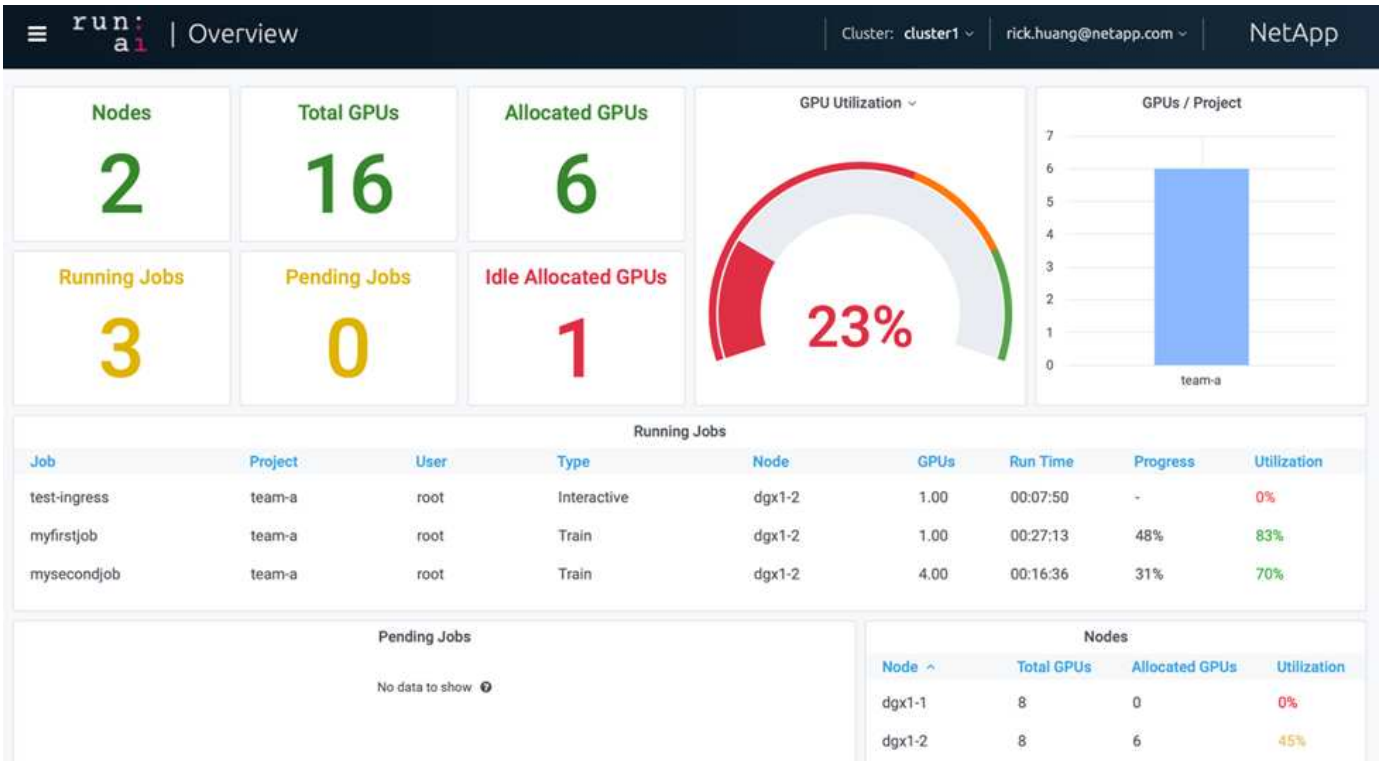
   ```
   kubectl apply -f runai-operator-<clustername>.yaml
   ```

4. 설치를 확인합니다.
   a. 로 이동합니다 "https://app.run.ai/".
   b. 개요 대시보드로 이동합니다.
   c. 오른쪽 위에 있는 GPU 수가 예상 GPU 수를 반영하고 GPU 노드가 모두 서버 목록에 있는지 확인합니다. 실행:AI 배포에 대한 자세한 내용은 을 참조하십시오 "Run 설치: 사내 Kubernetes 클러스터에 AI를 설치합니다" 및 "Run:AI CLI 설치".

"다음: AI 대시보드 및 보기를 실행합니다"

실행: AI 대시보드 및 뷰

Run:AI를 Kubernetes 클러스터에 설치하고 컨테이너를 올바르게 구성하면 에 다음과 같은 대시보드와 뷰가 표시됩니다 "https://app.run.ai" 다음 그림과 같이 브라우저에서



2개의 DGX-1 노드에서 제공하는 클러스터에 총 16개의 GPU가 있습니다. 노드 수, 총 사용 가능한 GPU, 워크로드와 할당된 할당된 할당된 GPU, 총 실행 중인 작업 수, 보류 중인 작업 및 유휴 할당 GPU를 볼 수 있습니다. 오른쪽의 막대 다이어그램에서는 프로젝트당 GPU를 보여 주며, 각 팀이 클러스터 리소스를 사용하는 방법을 요약합니다. 가운데는 작업 이름, 프로젝트, 사용자, 작업 유형, 각 작업이 실행 중인 노드, 해당 작업에 할당된 GPU 수, 작업의 현재 실행 시간, 작업의 작업 진행 상태, 해당 작업의 GPU 사용률 단일 팀('team-A')이 제출한 실행 중인 작업이 3개뿐이므로 클러스터 사용률이 낮은 상태(GPU 사용률이 23%)입니다.

다음 섹션에서는 프로젝트 탭에서 여러 팀을 생성하고 각 팀에 GPU를 할당하여 클러스터당 사용자가 많을 때 클러스터 사용을 최대화하고 리소스를 관리하는 방법을 보여줍니다. 테스트 시나리오는 훈련, 추론 및 대화형 워크로드 간에 메모리 및 GPU 리소스가 공유되는 엔터프라이즈 환경을 모방합니다.

"다음으로, 데이터 과학 팀을 위한 프로젝트 생성 및 GPU 할당"

데이터 과학 팀을 위한 프로젝트 생성 및 GPU 할당

연구원들은 Run:AI CLI, Kubeflow 또는 유사한 프로세스를 통해 워크로드를 제출할 수 있습니다. 리소스 할당을 간소화하고 우선 순위를 만들기 위해 Run:AI에는 프로젝트의 개념이 도입되었습니다. 프로젝트는 프로젝트 이름을 GPU 할당 및 기본 설정과 연결하는 할당량 요소입니다. 여러 데이터 과학 팀을 관리할 수 있는 간단하고 편리한 방법입니다.

워크로드를 제출하는 연구원은 프로젝트를 워크로드 요청과 연계해야 합니다. Run:AI 스케줄러는 요청을 현재 할당 및 프로젝트와 비교하여 워크로드에 리소스를 할당할 수 있는지 또는 보류 중 상태를 유지해야 하는지 여부를 결정합니다.

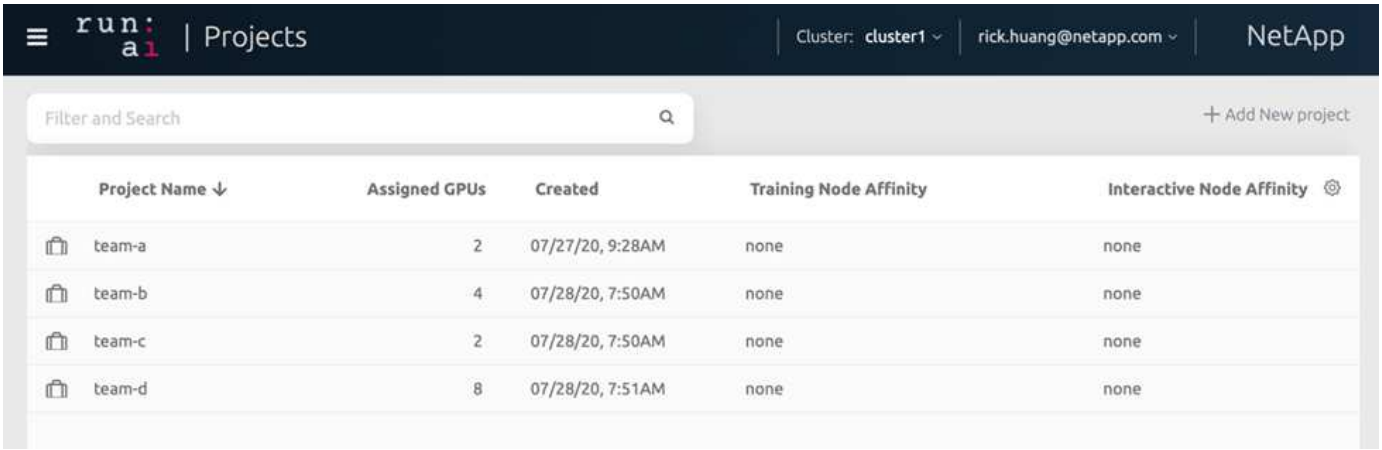
시스템 관리자는 실행: AI 프로젝트 탭에서 다음 매개 변수를 설정할 수 있습니다.

- * 모델 프로젝트 * 사용자별 프로젝트를 설정하고, 사용자 팀별로 프로젝트를 설정하고, 실제 조직 프로젝트별로 프로젝트를 설정합니다.
- * 프로젝트 할당량 * 각 프로젝트는 이 프로젝트에 동시에 할당할 수 있는 GPU 할당량과 연관됩니다. 이 프로젝트는 클러스터의 상태에 관계없이 이 프로젝트를 사용하는 연구원이 GPU 수를 확보할 수 있다는 점에서 보장된 할당량입니다. 일반적으로 프로젝트 할당의 합계는 클러스터에 있는 GPU 수와 같아야 합니다. 이 외에도 이

프로젝트의 사용자는 초과 할당량을 받을 수 있습니다. GPU를 사용하지 않는 한, 이 프로젝트를 사용하는 연구자는 더 많은 GPU를 얻을 수 있습니다. 에서는 할당량 초과 테스트 시나리오와 공정성 고려 사항을 보여 줍니다 "할당량 초과 GPU 할당을 통한 높은 클러스터 사용률 달성", "기본 자원 할당 공정성", 및 "할당량 초과 공정성".

- 새 프로젝트를 만들고, 기존 프로젝트를 업데이트하고, 기존 프로젝트를 삭제합니다.
- * 특정 노드 그룹에서 실행할 작업 제한 *. 특정 노드에서만 실행되도록 특정 프로젝트를 할당할 수 있습니다. 이 기능은 프로젝트 팀이 충분한 메모리를 갖춘 특수 하드웨어가 필요한 경우에 유용합니다. 또는 프로젝트 팀은 전문 예산으로 구입한 특정 하드웨어의 소유자가 되거나, 더 약한 하드웨어에서 작동하고 더 긴 훈련이나 무인 워크로드를 더 빠른 노드로 직접 처리하기 위해 직접 빌드하거나 대화형 워크로드를 실행해야 할 수도 있습니다. 노드를 그룹화하고 특정 프로젝트에 대한 선호도를 설정하는 명령은 을 참조하십시오 "AI 문서 를 실행하십시오".
- * 대화형 작업 기간 제한 *. 연구자들은 종종 대화식 작업을 종결하는 것을 잊어버립니다. 이로 인해 리소스가 낭비될 수 있습니다. 일부 조직에서는 대화형 작업의 기간을 제한하고 자동으로 작업을 종결하는 것을 선호합니다.

다음 그림에서는 네 개의 팀이 생성된 프로젝트 보기를 보여 줍니다. 각 팀에는 서로 다른 워크로드를 처리할 수 있는 서로 다른 수의 GPU가 할당되며, 총 GPU 수는 2개의 DGX-1로 구성된 클러스터에서 사용 가능한 총 GPU 수와 같습니다.



"다음: AI CLI 실행 에서 작업 제출"

실행 중인 작업 제출: AI CLI

이 섹션에서는 Kubernetes 작업 실행에 사용할 수 있는 기본 Run:AI 명령에 대한 자세한 정보를 제공합니다. 워크로드 유형에 따라 3개 부분으로 나뉩니다. AI/ML/DL 워크로드는 다음과 같은 두 가지 일반 유형으로 나눌 수 있습니다.

- * 무인 교육 세션 *. 이러한 유형의 워크로드를 사용하여 데이터 과학자는 자체 실행 워크로드를 준비하여 실행을 위해 보냅니다. 실행 중에 고객은 결과를 검토할 수 있습니다. 이러한 유형의 워크로드는 생산 또는 인물 개발에 사람의 개입이 필요 없는 단계에 있을 때 주로 사용됩니다.
- * 대화형 빌드 세션 *. 이러한 유형의 워크로드를 사용하여 데이터 과학자는 Bash, Jupyter Notebook, remote PyCharm 또는 유사한 IDE를 사용한 대화형 세션을 열고 GPU 리소스에 직접 액세스합니다. 연결된 포트를 사용하여 대화형 워크로드를 실행하는 세 번째 시나리오가 포함되어 컨테이너 사용자에게 내부 포트를 제공합니다.

무인 교육 워크로드

프로젝트를 설정하고 GPU를 지정한 후 명령줄에서 다음 명령을 사용하여 모든 Kubernetes 워크로드를 실행할 수 있습니다.

```
$ runai project set team-a runai submit hyper1 -i gcr.io/run-ai-
demo/quickstart -g 1
```

이 명령은 단일 GPU를 할당하여 팀 A의 무인 교육 작업을 시작합니다. 이 작업은 샘플 Docker 이미지 'GCR.IO/RUN-AI-DEMO/QuickStart'를 기반으로 합니다. 우리는 그 일을 하이퍼1이라고 명명했다. 그런 다음 다음 다음 명령을 실행하여 작업의 진행률을 모니터링할 수 있습니다.

```
$ runai list
```

다음 그림은 루나이 리스트 명령의 결과를 보여준다. 표시되는 일반적인 상태는 다음과 같습니다.

- 'ContainerCreating' Docker 컨테이너를 클라우드 저장소에서 다운로드하고 있습니다.
- '보류 중'. 작업이 예약될 때까지 대기 중입니다.
- '러닝'입니다. 작업이 실행 중입니다.

```
~> runai list
Showing jobs for project team-a
NAME    STATUS   AGE  NODE                                  IMAGE                        TYPE        PROJECT  USER   GPUs
hyper1  Running  11s  gke-dev-yaron1-gpu-4-pool-154f511d-5nk5  gcr.io/run-ai-demo/quickstart  Train       team-a   yaron  1
```

작업에 대한 추가 상태를 가져오려면 다음 명령을 실행합니다.

```
$ runai get hyper1
```

작업의 로그를 보려면 "runai logs <job-name>" 명령을 실행합니다.

```
$ runai logs hyper1
```

이 예에서는 각 단계에 대해 현재 교육 Epoch, ETA, 손실 함수 값, 정확도 및 경과 시간을 포함하여 실행 중인 DL 세션의 로그를 확인해야 합니다.

의 Run:AI UI에서 클러스터 상태를 볼 수 있습니다 "https://app.run.ai/". 대시보드 > 개요 에서 GPU 사용률을 모니터링할 수 있습니다.

이 워크로드를 중지하려면 다음 명령을 실행합니다.

```
$ runai delte hyper1
```

이 명령은 교육 워크로드를 중지합니다. 이 작업은 'runai list'를 다시 실행하여 확인할 수 있습니다. 자세한 내용은 을 참조하십시오 "무인 교육 워크로드 실행".

대화형 빌드 워크로드

프로젝트를 설정하고 GPU를 할당하면 명령줄에서 다음 명령을 사용하여 대화형 빌드 워크로드를 실행할 수 있습니다.

```
$ runai submit build1 -i python -g 1 --interactive --command sleep --args
infinity
```

이 작업은 샘플 Docker 이미지 Python을 기반으로 합니다. 우리는 작업 구축1이라는 이름을 붙였습니다.

> 인터액티브 플래그는 작업이 시작이나 끝이 없다는 뜻입니다 이 일을 마무리하는 것은 연구자의 책임입니다. 관리자는 시스템에 의해 종료된 후 대화형 작업에 대한 시간 제한을 정의할 수 있습니다.

이 작업에는 '--g 1' 플래그가 GPU를 하나만 할당합니다. 명령어와 논리는 '--명령 슬립—args 무한대'입니다. 명령을 제공해야 합니다. 그렇지 않고 컨테이너가 시작되고 즉시 종료됩니다.

다음 명령은 에 설명된 명령과 유사하게 작동합니다 [Unattended Training Workloads]:

- 'runai list': 이름, 상태, 나이, 노드, 이미지, 작업을 위해 프로젝트, 사용자 및 GPU를 지원합니다.
- runai get build1: 작업 build1에 추가 상태를 표시합니다.
- 'runai delete build1': 대화형 워크로드 빌드 중지1. bash 셸을 컨테이너에 가져오려면 다음 명령을 사용합니다.

```
$ runai bash build1
```

그러면 컴퓨터에 직접 셸이 제공됩니다. 그런 다음 데이터 과학자는 컨테이너 내에서 모델을 개발 또는 미세 조정할 수 있습니다.

의 Run:AI UI에서 클러스터 상태를 볼 수 있습니다 "https://app.run.ai". 자세한 내용은 을 참조하십시오 "대화형 빌드 워크로드 시작 및 사용".

연결된 포트를 사용하는 대화형 작업 부하

대화형 빌드 워크로드의 확장으로 Run:AI CLI로 컨테이너를 시작할 때 컨테이너 사용자에게 내부 포트를 표시할 수 있습니다. 이 기능은 Jupyter Notebooks와 함께 작업하거나 다른 마이크로서비스에 연결하는 클라우드 환경에 유용합니다. "침투" Kubernetes 클러스터 외부에서 Kubernetes 서비스에 액세스할 수 있습니다. 어떤 인바운드 연결이 어떤 서비스에 연결할지 정의하는 규칙 모음을 만들어 액세스를 구성할 수 있습니다.

클러스터의 서비스에 대한 외부 액세스를 보다 효율적으로 관리하기 위해 클러스터 관리자를 설치하는 것이 좋습니다 "침투" 및 로드 밸런서를 구성합니다.

서비스 유형으로 수신을 사용하려면 다음 명령을 실행하여 워크로드를 제출할 때 메서드 유형과 포트를 설정합니다.

```
$ runai submit test-ingress -i jupyter/base-notebook -g 1 \
  --interactive --service-type=ingress --port 8888 \
  --args="--NotebookApp.base_url=test-ingress" --command=start-notebook.sh
```

컨테이너가 성공적으로 시작된 후 runai list를 실행하여 Jupyter Notebook에 액세스할 수 있는 Service URL(S)을 확인합니다. URL은 수신 엔드포인트, 작업 이름 및 포트로 구성됩니다. 예를 들어 를 참조하십시오 https://10.255.174.13/test-ingress-8888.

자세한 내용은 을 참조하십시오 "연결된 포트를 사용하여 대화형 빌드 워크로드 시작".

"다음: 높은 클러스터 사용률 달성"

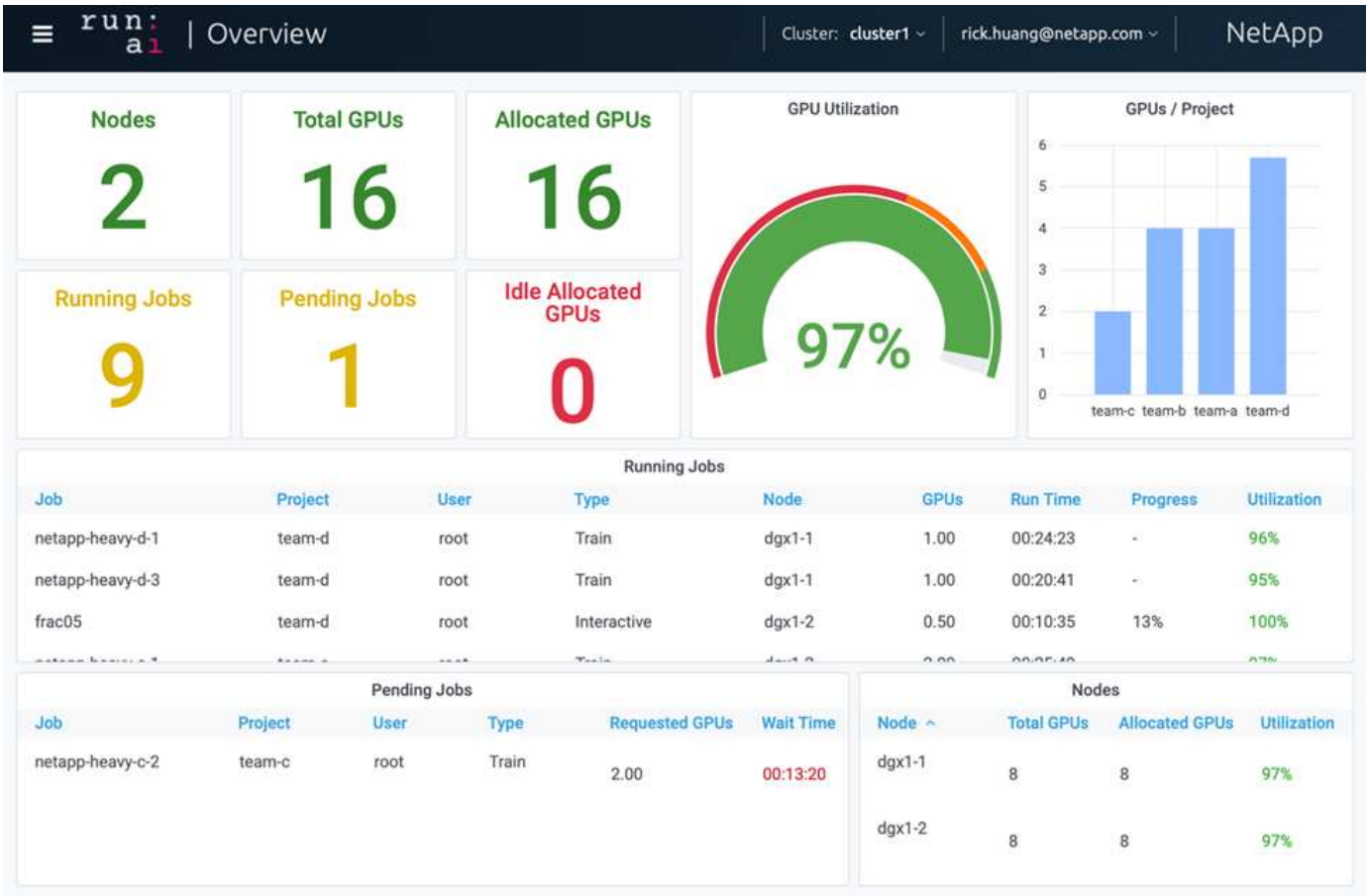
높은 클러스터 사용률 달성

이 섹션에서는 GPU 리소스의 우선순위 지정 및 밸런싱을 유지하면서 높은 클러스터 사용률을 달성하는 Run:AI 오케스트레이션 솔루션을 시연하기 위해 4개의 데이터 과학 팀이 각자 고유의 워크로드를 제출하는 실제 시나리오를 에뮬레이트합니다. 먼저 섹션에 설명된 ResNet-50 벤치마크를 사용합니다 "ImageNet 데이터 세트 벤치마크 요약을 통한 ResNet-50":

```
$ runai submit netapp1 -i netapp/tensorflow-tf1-py3:20.01.0 --local-image
--large-shm  -v /mnt:/mnt -v /tmp:/tmp --command python --args
"/netapp/scripts/run.py" --args "--
dataset_dir=/mnt/mount_0/dataset/imagenet/imagenet_original/" --args "--
num_mounts=2"  --args "--dgx_version=dgx1" --args "--num_devices=1" -g 1
```

에서와 같이 ResNet-50 벤치마크를 실행했습니다 "NVA-1121". 우리는 공공 Docker 리포지토리에 없는 컨테이너에 '--local-image' 플래그를 사용했습니다. 호스트 DGX-1 노드의 /mnt와 /tmp 디렉토리를 각각 컨테이너에 `/mnt', '/tmp' 디렉토리에 마운트했습니다. 데이터 세트는 디렉토리를 가리키는 dataset_dir와 함께 NetApp AFFA800에 있습니다. '--num_devices=1'과 '-g 1'은 이 작업에 하나의 GPU를 할당한다는 것을 의미합니다. 전자는 run.py 스크립트의 주장이고 후자는 runai submit 명령의 플래그입니다.

다음 그림은 97%의 GPU 사용률과 16개의 사용 가능한 GPU가 할당된 시스템 개요 대시보드를 보여 줍니다. GPU/프로젝트 막대 차트에서 각 팀에 할당된 GPU 수를 쉽게 확인할 수 있습니다. 실행 중인 작업 창에는 현재 실행 중인 작업 이름, 프로젝트, 사용자, 유형, 노드, GPU 사용량, 실행 시간, 진행률 및 활용률 세부 정보 대기 시간이 있는 대기열의 워크로드 목록이 보류 중인 작업에 표시됩니다. 마지막으로, 노드 상자는 클러스터의 개별 DGX-1 노드에 대한 GPU 수와 활용률을 제공합니다.



"다음으로, 덜 까다롭거나 대화형 워크로드에 대한 GPU 할당 분수를 지정합니다"

덜 까다로운 워크로드 또는 대화형 워크로드에 대한 부분 **GPU** 할당

연구자와 개발자가 개발, 고매개 변수 조정 또는 디버깅 단계에서 자신의 모델을 작업할 때 이러한 워크로드는 일반적으로 컴퓨팅 리소스를 적게 사용합니다. 따라서 동일한 GPU를 다른 워크로드에 동시에 할당할 수 있도록 소수점 GPU 및 메모리를 프로비저닝하는 것이 더 효율적입니다. 실행: AI의 오케스트레이션 솔루션은 Kubernetes에서 컨테이너화된 워크로드를 위한 분할 GPU 공유 시스템을 제공합니다. 이 시스템은 CUDA 프로그램을 실행하는 워크로드를 지원하며 추론과 모델 구축과 같은 가벼운 AI 작업에 특히 적합합니다. 소수점 GPU 시스템은 데이터 과학과 AI 엔지니어링 팀에게 단일 GPU에서 동시에 여러 워크로드를 실행할 수 있는 기능을 투명하게 제공합니다. 이를 통해 기업은 컴퓨터 비전, 음성 인식 및 자연어 처리와 같은 더 많은 워크로드를 동일한 하드웨어에서 실행할 수 있으므로 비용이 절감됩니다.

실행: AI의 분할 GPU 시스템은 컨테이너가 자급식 프로세서인 것처럼 사용하고 액세스할 수 있는 자체 메모리 및 컴퓨팅 공간을 사용하여 가상화된 논리 GPU를 효과적으로 생성합니다. 따라서 여러 워크로드가 서로 간섭하지 않고 동일한 GPU의 컨테이너에서 나란히 실행될 수 있습니다. 이 솔루션은 투명하고 단순하며 이식 가능하며 컨테이너 자체를 변경할 필요가 없습니다.

일반적인 UseCase는 동일한 GPU에서 실행되는 작업을 2~8개 볼 수 있으며, 이는 동일한 하드웨어에서 8배 더 많은 작업을 수행할 수 있음을 의미합니다.

다음 그림에서 PROJECT 팀 d에 속한 Frac05 작업에 대해 할당된 GPU 수가 0.50인 것을 알 수 있다. 이는 컨테이너에 사용 가능한 GPU 메모리가 DGX-1 노드의 V100 GPU당 32GB의 절반 인 16,255MB임을 보여 주는 'NVIDIA-SMI' 명령으로 더욱 검증되었습니다.

```
root@run-deploy:~# runai bash frac05 -p team-d
root@frac05-0:/workload# nvidia-smi
Tue Jul 28 15:17:03 2020
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 450.51.05    Driver Version: 450.51.05    CUDA Version: 11.0     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2...  On   | 00000000:07:00.0 Off |                    0 |
| N/A   57C    P0   240W / 300W |  15525MiB / 16255MiB |     100%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A       156      C   python3                         15525MiB |
+-----------------------------------------------------------------------------+
```

"다음으로, 초과 할당 GPU 할당을 사용하여 높은 클러스터 사용률을 달성하십시오"

할당량 초과 GPU 할당을 통한 높은 클러스터 사용률 달성

섹션을 참조하십시오 "기본 자원 할당 공정성", 및 "할당량 초과 공정성"복잡한 워크로드 관리, 자동 사전 예방 예약 및 초과 할당량 GPU 프로비저닝을 위한 Run:AI 조정 기능을 시연하기 위해 고급 테스트 시나리오를 고안했습니다. 이를 통해 ONTAP AI 환경에서 클러스터 리소스를 많이 사용하고 엔터프라이즈급 데이터 과학 팀 생산성을 최적화할 수 있었습니다.

이 세 섹션에서는 다음 프로젝트 및 할당량을 설정합니다.

프로젝트	할당량
팀-A	4
팀-b	2
팀 - c	2
팀 d	8

또한 다음 세 개의 단원에 다음과 같은 컨테이너를 사용합니다.

- Jupyter Notebook: jupyter/base-notebook
- Run:AI QuickStart:'GCR.IO/RUN-AI-DEMO/QuickStart'를 실행하십시오

이 테스트 시나리오에 대해 다음과 같은 목표를 설정했습니다.

- 리소스 프로비저닝의 간편성 및 리소스를 사용자로부터 추상화한 방법을 보여줍니다
- GPU의 분수와 GPU의 정수 수를 간편하게 프로비저닝하는 방법을 보여줍니다
- 클러스터에 무료 GPU가 있을 경우 팀 또는 사용자가 리소스 할당량을 처리할 수 있으므로 시스템에서 컴퓨팅 병목 현상이 해소되는 방법을 보여줍니다
- NetApp 컨테이너와 같은 컴퓨팅 집약적인 작업을 실행할 때 NetApp 솔루션을 사용하여 데이터 파이프라인의 병목 현상을 제거하는 방법을 보여줍니다
- 시스템을 사용하여 여러 유형의 컨테이너를 실행하는 방법을 보여 줍니다
  - Jupyter 노트북
  - 실행: AI 컨테이너
- 클러스터가 가득 찼을 때 높은 사용률을 표시합니다

테스트 중에 실행된 실제 명령 시퀀스에 대한 자세한 내용은 을 참조하십시오 "섹션 4.8의 테스트 세부 사항".

13개의 워크로드를 모두 제출하면 다음 그림과 같이 할당된 컨테이너 이름 및 GPU 목록을 볼 수 있습니다. NetApp은 7개의 교육 및 6개의 대화식 작업을 통해 4개의 데이터 과학 팀을 시뮬레이션하며 각 팀은 개발 또는 자체 모델을 실행하고 있습니다. 대화형 작업의 경우, 개별 개발자는 Jupyter Notebooks를 사용하여 코드를 작성하거나 디버깅합니다. 따라서 클러스터 리소스를 너무 많이 사용하지 않고 GPU 분할을 프로비저닝하는 것이 좋습니다.

```
root@run-deploy:~# runai list -A
NAME            STATUS   AGE  NODE    IMAGE                          TYPE        PROJECT  USER  GPUs  CREATED BY CLI  SERVICE URL(S)
b-4-gg          Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-b   root  2     true
c-5-g           Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-c   root  1     true
c-4-gg          Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-c   root  2     true
b-3-g           Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-b   root  1     true
c-3-g02         Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.2   true
d-1-gggg        Running  2m   dgx1-2  gcr.io/run-ai-demo/quickstart  Train       team-d   root  4     true
c-2-g03         Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.3   true
c-1-g05         Running  2m   dgx1-1  gcr.io/run-ai-demo/quickstart  Interactive team-c   root  0.5   true
a-2-gg          Running  3m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-a   root  2     true
b-2-g04         Running  3m   dgx1-2  gcr.io/run-ai-demo/quickstart  Interactive team-b   root  0.4   true
a-1-g           Running  3m   dgx1-1  gcr.io/run-ai-demo/quickstart  Train       team-a   root  1     true
b-1-g06         Running  3m   dgx1-2  gcr.io/run-ai-demo/quickstart  Interactive team-b   root  0.6   true
a-1-1-jupyter   Running  3m   dgx1-1  jupyter/base-notebook          Interactive team-a   root  1     true            http://10.61.218.134/a-1-1-jupyter,
https://10.61.218.134/a-1-1-jupyter
```

이 테스트 시나리오의 결과는 다음과 같습니다.

- 클러스터가 꽉 찼어야 합니다. 16/16개의 GPU를 사용했습니다.
- 높은 클러스터 사용률.
- 부분 할당으로 인해 GPU보다 더 많은 실험
- 팀 d는 쿼터를 모두 사용하지 않으므로 팀 b와 팀 c는 실험에 추가 GPU를 사용할 수 있어 혁신의 시간을 단축할 수 있습니다.

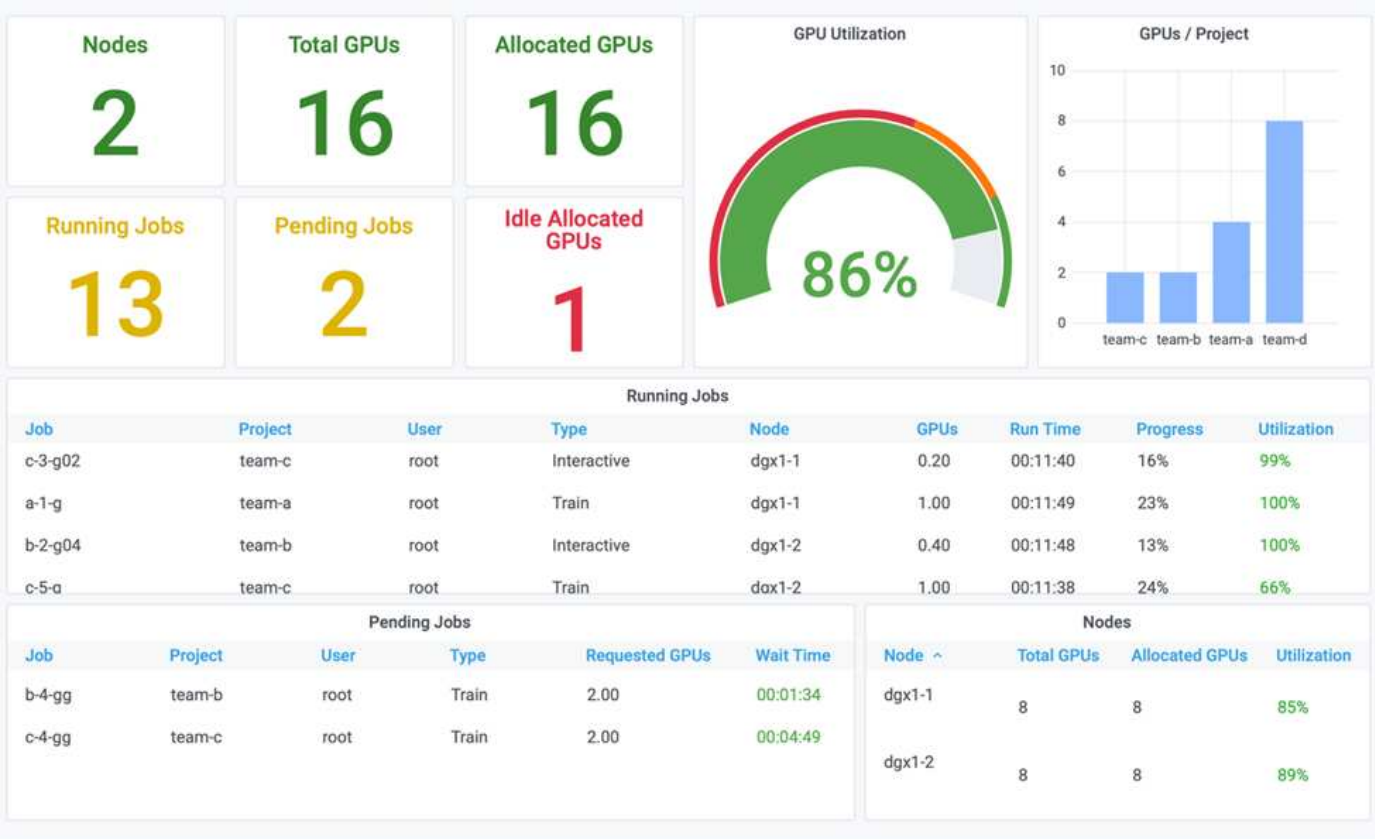
"다음: 기본 자원 할당 공정성"

기본 자원 할당 공정성

이 섹션에서는 'team-d'가 더 많은 GPU(할당량 아래)를 요청할 때 시스템이 'team-b'와 'team-c'의 워크로드를 일시 중지하고 공평한 분배 방식으로 보류 중인 상태로 전환한다는 것을 보여 줍니다.

작업 제출, 사용된 컨테이너 이미지 및 실행된 명령 시퀀스를 포함한 자세한 내용은 섹션을 참조하십시오 "섹션 4.9의 테스트 세부 정보".

다음 그림은 자동 로드 밸런싱 및 사전 예방 예약 기능으로 인해 발생하는 클러스터 활용률, 팀당 할당된 GPU 및 보류 중인 작업을 보여줍니다. 모든 팀 작업 부하에 의해 요청된 총 GPU 수가 클러스터에서 사용 가능한 총 GPU 수를 초과할 때 Run:AI의 내부 공정성 알고리즘은 프로젝트 할당량을 충족했기 때문에 "team-b"와 "team-c"에 대해 각각 하나의 작업을 일시 중지한다는 것을 알 수 있습니다. 따라서 전반적인 높은 클러스터 활용률이 제공되지만 데이터 과학 팀은 관리자가 설정한 리소스 제약 조건에서 작업을 계속 수행할 수 있습니다.



이 테스트 시나리오의 결과는 다음과 같습니다.

- * 자동 로드 밸런싱. * 시스템은 GPU의 할당량을 자동으로 조정하여 각 팀에서 현재 할당량을 사용하고 있습니다. 일시 중지된 워크로드는 할당량이 초과된 팀에 속합니다.
- * 공정한 공유 일시 중지. * 시스템이 할당량이 초과된 팀의 작업 부하를 중지하도록 선택한 다음 다른 팀의 작업 부하를 중지시킵니다. 실행: AI에는 내부 공정성 알고리즘이 있습니다.

"다음: 할당량 초과 공정성"

할당량 초과 공정성

이 섹션에서는 여러 팀에서 워크로드를 제출하고 할당량을 초과하는 시나리오를 확장합니다. 이 방법으로 Run:AI의 Fairness 알고리즘이 사전 설정된 할당량의 비율에 따라 클러스터 리소스를 할당하는 방법을 보여 줍니다.

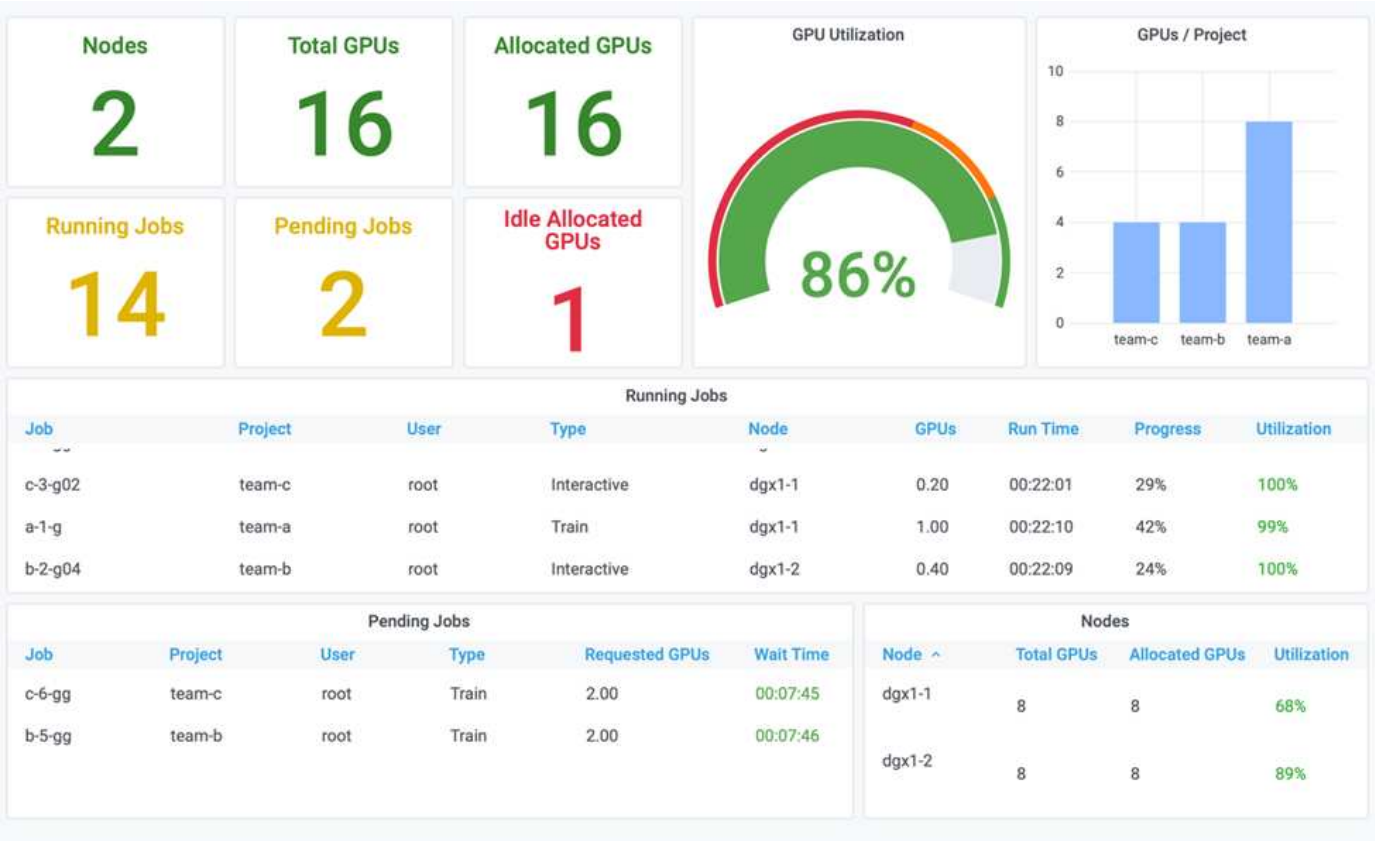
이 테스트 시나리오의 목표:

- 여러 팀에서 할당량을 통해 GPU를 요청할 때 큐 메커니즘을 표시합니다.
- 할당량 비율에 따라 할당량이 초과된 여러 팀 간에 클러스터가 공평하게 분배되어 할당량이 더 큰 팀이 여유 용량을 더 많이 점유하도록 하는 방법을 보여 줍니다.

의 끝에 있습니다 "기본 자원 할당 공정성"팀-b, 팀-c의 두 가지 워크로드가 대기 중입니다. 이 섹션에서는 추가 워크로드를 전담합니다.

작업 제출, 사용된 컨테이너 이미지 및 실행된 명령 시퀀스를 포함한 자세한 내용은 을 참조하십시오 "섹션 4.10의 테스트 세부 정보".

섹션에 따라 모든 작업이 제출되는 경우 "섹션 4.10의 테스트 세부 정보"시스템 대시보드에는 팀-A, 팀-b, 팀-c가 모두 미리 설정된 할당량보다 더 많은 GPU를 가지고 있는 것으로 표시됩니다. 팀 A는 미리 설정된 소프트 쿼터보다 4개의 GPU를 더 점유하고, 팀 b와 팀 c는 각각 소프트 할당량(2개)보다 2개의 GPU를 더 점유합니다. 할당된 초과 할당량 GPU의 비율은 사전 설정된 할당량의 비율과 동일합니다. 이는 시스템이 사전 설정 할당량을 우선 순위에 따라 사용하고 여러 팀에서 더 많은 GPU를 요청하고 할당량을 초과할 경우 적절히 프로비저닝하기 때문입니다. 이러한 자동 로드 밸런싱은 엔터프라이즈 데이터 과학 팀이 AI 모델 개발 및 생산에 적극적으로 참여할 때 공정성과 우선순위를 제공합니다.



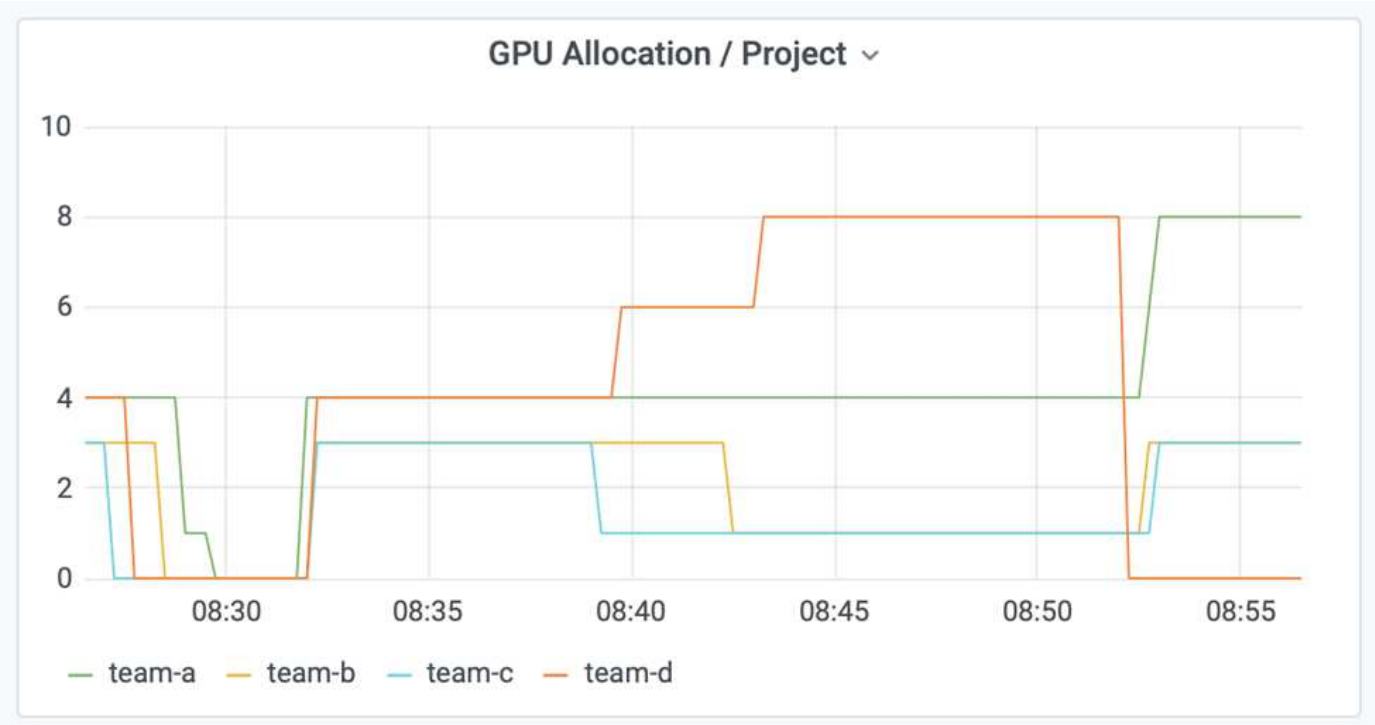
이 테스트 시나리오의 결과는 다음과 같습니다.

- 시스템이 다른 팀의 작업 부하를 취소하기 시작합니다.
- 대기열은 공정성 알고리즘에 따라 결정되며, 팀-b와 팀-c는 할당량 초과 GPU(할당량이 비슷하므로)와 동일한 양을 할당받습니다. 또 팀-A는 쿼터보다 2배 많은 양의 GPU를 갖게 된다. 팀-b, 팀-c의 쿼터보다 쿼터량이 2배 더 높기 때문이다.
- 모든 할당이 자동으로 수행됩니다.

따라서 시스템은 다음 상태에서 안정되어야 합니다.

프로젝트	**GPU**가 할당되었습니다	설명
팀-A	8월 4일	할당량에 4개의 GPU가 사용됩니다. 대기열이 비어 있습니다.
팀-b	4월 2일	할당량을 통해 2개의 GPU가 제공됩니다. 하나의 워크로드가 대기 중입니다.
팀 - c	4월 2일	할당량을 통해 2개의 GPU가 제공됩니다. 하나의 워크로드가 대기 중입니다.
팀 d	0/8	GPU를 전혀 사용하지 않고, 대기 중인 워크로드가 없습니다.

다음 그림에서는 섹션에 대한 Run:AI Analytics 대시보드의 시간별 프로젝트당 GPU 할당을 보여 줍니다 "할당량 초과 GPU 할당을 통한 높은 클러스터 사용률 달성", "기본 자원 할당 공정성", 및 "할당량 초과 공정성". 그림의 각 줄은 언제든지 특정 데이터 과학 팀에 프로비저닝된 GPU 수를 나타냅니다. 시스템이 제출된 워크로드에 따라 GPU를 동적으로 할당하는지 확인할 수 있습니다. 따라서 클러스터에 사용 가능한 GPU가 있을 때 팀은 할당량을 초과하고, 공정성에 따라 작업을 사전에 미분하여 4팀 모두에 대해 안정적인 상태가 될 수 있습니다.



"다음: Trident에서 프로비저닝한 PersistentVolume에 데이터 저장"

Trident에서 프로비저닝한 **PersistentVolume**에 데이터 저장

NetApp Trident는 컨테이너화된 애플리케이션의 정교한 지속성 요구사항을 충족하도록 설계된 완전 지원되는 오픈 소스 프로젝트입니다. 데이터 계층화, 암호화, NetApp Snapshot 기술, 규정 준수 및 NetApp ONTAP 데이터 관리 소프트웨어에서 제공하는 우수한 성능의 이점을 추가로 활용하여 Trident에서 프로비저닝한 Kubernetes PersistentVolume(PV)에 데이터를 읽고 쓸 수 있습니다.

기존 네임스페이스에서 **PVC** 재사용

대규모 AI 프로젝트의 경우, 여러 컨테이너가 동일한 Kubernetes PV에서 데이터를 읽고 쓰는 것이 더 효율적일 수 있습니다. Kubernetes PVC(Persistent Volume Claim)를 재사용하려면 이미 PVC를 생성해야 합니다. 를 참조하십시오 "NetApp Trident 문서" PVC 작성에 대한 자세한 내용은. 다음은 기존 PVC를 재사용하는 예입니다.

```
$ runai submit pvc-test -p team-a --pvc test:/tmp/pvc1mount -i gcr.io/run-
ai-demo/quickstart -g 1
```

다음 명령어를 실행해 프로젝트 팀 A에 대한 작업 PVC-TEST의 상태를 확인할 수 있다.

```
$ runai get pvc-test -p team-a
```

team-A job 'PVC-test'에 PV/tmp/pvc1mount가 마운트된 것을 볼 수 있습니다. 이렇게 하면 여러 컨테이너를 동일한 볼륨에서 읽을 수 있으므로 개발 또는 운영 중인 경쟁 모델이 여러 개 있을 때 유용합니다. 데이터 과학자는 모델의 앙상블을 만든 다음 대부분의 투표 또는 기타 기술을 통해 예측 결과를 결합할 수 있습니다.

다음을 사용하여 컨테이너 셸에 액세스합니다.

```
$ runai bash pvc-test -p team-a
```

그런 다음 마운트된 볼륨을 확인하고 컨테이너 내의 데이터에 액세스할 수 있습니다.

PVC를 재사용할 수 있는 이 기능은 NetApp FlexVol 볼륨 및 NetApp ONTAP FlexGroup 볼륨과 함께 작동하여 데이터 엔지니어가 보다 유연하고 강력한 데이터 관리 옵션을 통해 NetApp이 제공하는 Data Fabric을 활용할 수 있도록 지원합니다.

"다음: 결론"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.